

ROS 1일차 과제2 보고서, 김도훈, 2026406041

<목차>

1. 과제2 설명
2. 과제2 구현
3. 실행 결과

1. 과제2 설명

과제2는

2. 과제2 구현

사용한 Topic은 다음과 같다.

topic_string
topic_int32
topic_float32
topic_bool

C++ Publisher

C++에서는 rclcpp를 사용하여 Publisher를 생성하였다. 각 Topic마다 메시지 타입을 지정하고 0.5초마다 데이터를 발행하도록 Timer를 설정하였다.

```
publisher_string_ = create_publisher<std_msgs::msg::String>("topic_string", 10);  
publisher_int32_ = create_publisher<std_msgs::msg::Int32>("topic_int32", 10);
```

Callback 함수에서는 메시지에 값을 저장한 뒤 publish()를 이용해 전송하였다.

```
message.data = "Hello, world! " + std::to_string(count_);  
message2.data = count_;  
publisher_string_->publish(message);  
publisher_int32_->publish(message2);
```

같은 방식으로 Float32, Bool 타입의 데이터도 각각 발행하도록 구현하였다.

C++ Subscriber

C++ Subscriber에서는 create_subscription()을 이용하여 각 Topic을 구독하였다.

```
subscription_string_ = create_subscription<std_msgs::msg::String>("topic_string", 10,  
std::bind( &MinimalSubscriber::topic_callback_string, this, _1));
```

Publisher에서 메시지가 전달되면 Callback 함수가 실행되고 msg.data를 이용하여 받은 값을 출력하였다.

```
void topic_callback_string( const std_msgs::msg::String & msg) const
{
    RCLCPP_INFO( get_logger(), "String subscriber: '%s'", msg.data.c_str());
}
```

Int32, Float32, Bool도 동일한 방식으로 각각의 Callback 함수를 만들어 데이터를 수신하였다.

Python Publisher

Python에서는 ROS2 Python 라이브러리인 rclpy를 사용하였다.

```
self.publisher_string = self.create_publisher( String, 'topic_string', 10)
self.publisher_int32 = self.create_publisher( Int32, 'topic_int32', 10)
```

Timer Callback에서 메시지 데이터를 설정한 후 publish()를 이용하여 전송하였다.

```
message1 = String()
message1.data = 'Hello, world! ' + str(self.count)
message2 = Int32()
message2.data = self.count
self.publisher_string.publish(message1)
self.publisher_int32.publish(message2)
```

C++ Publisher와 동일하게 Float32, Bool 메시지도 추가하여 총 네 가지 데이터를 발행하도록 구현하였다.

Python Subscriber

Python Subscriber에서는 create_subscription()을 이용하여 Topic을 구독하였다.

```
self.subscription_string=self.create_subscription(String,'topic_string',
self.topic_callback_string, 10)
```

메시지가 수신되면 연결된 Callback 함수가 실행되고 전달받은 값을 출력하였다.

```
def topic_callback_string(self, msg):
    self.get_logger().info( f"String subscriber: '{msg.data}'")
```

Int32, Float32, Bool 역시 같은 방식으로 각각 구독하도록 구현하였다.

C++과 Python 간 통신

C++과 Python에서 사용하는 문법은 다르지만 Topic 이름과 ROS2 메시지 타입을 동일하게 설정하였기 때문에 서로 통신할 수 있었다. 예를 들어 C++에서는 `std_msgs::msg::String` 을 사용하고 Python에서는 `String` 을 사용하지만 ROS2에서는 둘 다 `std_msgs/msg/String` 타입으로 처리된다. 따라서 다음 네 가지 통신을 모두 구현할 수 있었다.

```
C++ Publisher    -> C++ Subscriber
Python Publisher -> Python Subscriber
C++ Publisher    -> Python Subscriber
Python Publisher -> C++ Subscriber
```

이를 통해 ROS2에서는 서로 다른 프로그래밍 언어로 작성된 Node라도 Topic 이름과 메시지 타입이 같으면 통신할 수 있음을 확인하였다.

3. 실행 결과

스크린샷 오류가 있어 영상으로 대체하겠습니다. 죄송합니다.

1. C++ Publisher -> C++ Subscriber

C++의 `rclcpp`를 이용해 Publisher와 Subscriber를 구현하였다.

Publisher가 `String`, `Int32`, `Float32`, `Bool` 데이터를 각각의 Topic으로 발행하고, C++ Subscriber가 같은 Topic과 메시지 타입을 구독하여 데이터를 수신한다.

2. Python Publisher -> Python Subscriber

Python Publisher가 여러 자료형의 메시지를 Topic으로 발행하고 Python Subscriber가 Callback 함수를 통해 데이터를 받아 터미널에 출력한다.

3. Python Publisher -> C++ Subscriber

Python과 C++은 서로 다른 언어이지만 Topic 이름과 ROS2 메시지 타입을 동일하게 설정하면 정상적으로 통신할 수 있음을 확인하였다.

4. C++ Publisher -> Python Subscriber

C++ Publisher가 발행한 `String`, `Int32`, `Float32`, `Bool` 데이터를 Python Subscriber가 수신한다.

